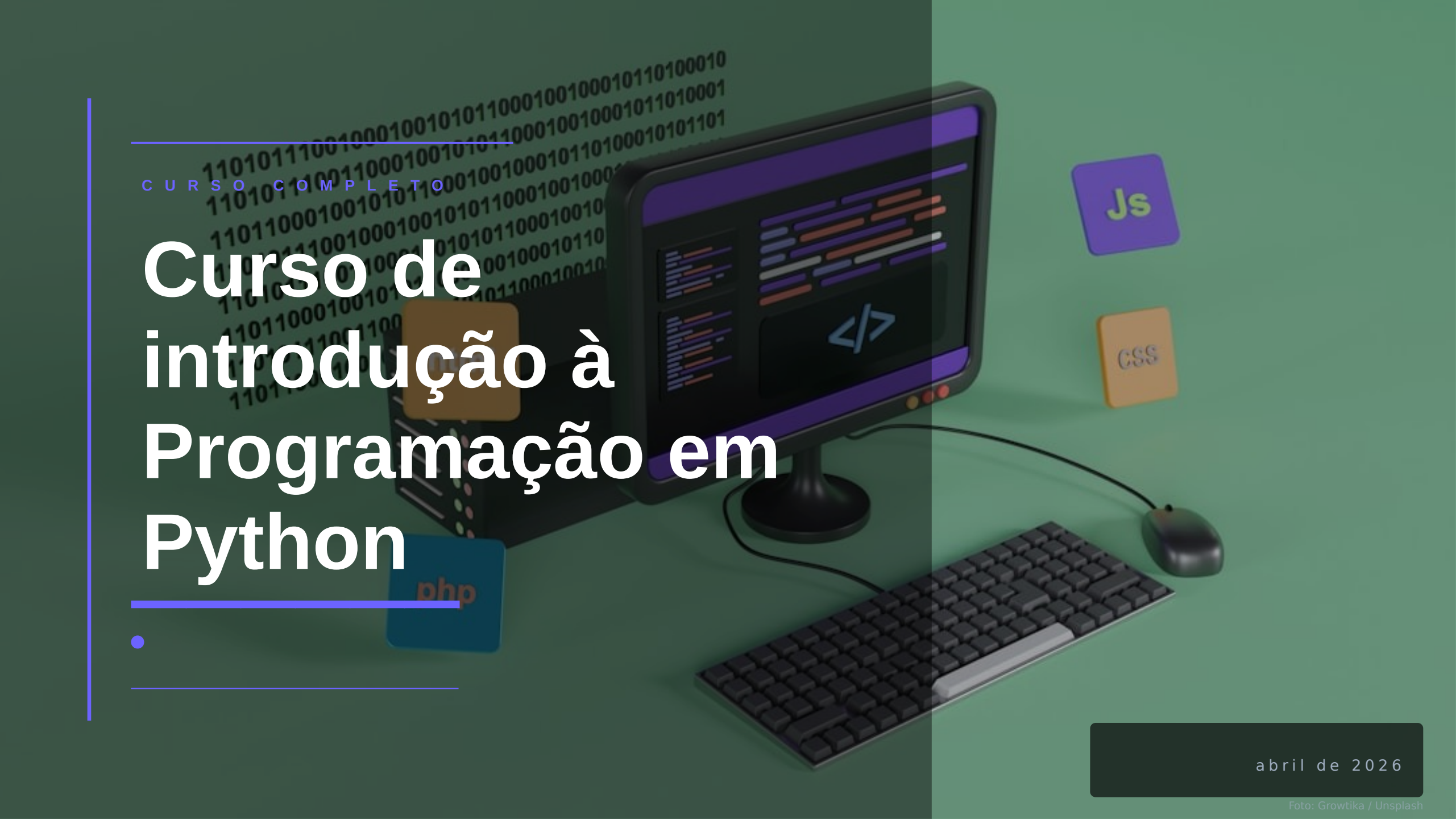
CURSO COMPLETO

Curso de introdução à Programação em Python



abril de 2026

Foto: Growtika / Unsplash

CONTEÚDO PROGRAMÁTICO

Índice · 1/2

1	**Primeiros Passos com Python e Fundamentos da Programação**	Compreender os princípios fundamentais da programação e o papel de Python. Configurar um ambiente de desenvolvimento eficiente para programação em Python. Criar e executar seu...
2	**Estruturas de Dados Essenciais e Manipulação**	Compreender as características e funcionalidades das estruturas de dados básicas em Python. Desenvolver a capacidade de manipular listas, tuplas, conjuntos e dicionários. Aplicar...
3	**Funções, Módulos e Organização de Código**	Compreender a importância de funções, módulos e boa organização do código. Desenvolver a capacidade de criar e utilizar funções eficientes em Python. Aplicar princípios de...
4	**Manipulação de Arquivos e Formatos de Dados (CSV, JSON)**	Compreender a importância da manipulação de arquivos para a persistência de dados em aplicações Python. Utilizar os módulos csv e json do Python para ler e gravar dados em...
5	**Programação Orientada a Objetos (POO) com Python**	Entender os conceitos fundamentais da Programação Orientada a Objetos (POO). Aprender a criar classes e objetos em Python, definindo seus atributos e métodos. Aplicar os...
6	**Tratamento de Exceções e Geração de Logs**	Aplicar o tratamento de exceções para criar softwares robustos e tolerantes a falhas. Utilizar a geração de logs para depurar, monitorar e manter aplicações Python. Desenvolver a...

CONTEÚDO PROGRAMÁTICO

Índice · 2/2

7 **Testes Automatizados com `unittest` e Boas Práticas**

Dominar o uso do módulo unittest para criar testes automatizados eficazes, cobrindo cenários como validação de APIs, lógica de negócios complexa e interação com banco de dados...

8 **Projeto Final: Construindo e Distribuindo um Utilitário Python**

Projetar e codificar um utilitário Python funcional do zero. Empacotar e gerenciar dependências de projetos Python profissionalmente. Publicar e distribuir utilitários Python no...

MÓDULO 1

Primeiros Passos com Python e Fundamentos da Programação

O QUE VOCÊ VAI APRENDER

- Configurar um ambiente de desenvolvimento eficiente para programação em Python.
- Criar e executar seu primeiro programa Python com sucesso.
- A programação é a arte e a ciência de fornecer instruções a um computador para executar tarefas espe

Foto: Janine Dominoes / Unsplash

Primeiros Passos com Python e Fundamentos da Programação

1. Configurar um ambiente de desenvolvimento eficiente para programação em Python.
2. Criar e executar seu primeiro programa Python com sucesso.
3. A programação é a arte e a ciência de fornecer instruções a um computador para executar tarefas específicas.
4. Um programa de computador é uma sequência lógica de instruções que o computador pode entender.

Primeiros Passos com Python e Fundamentos da Programação

1

O objetivo é traduzir problemas do mundo real em uma linguagem que o computador possa processar e resolver.

2

Linguagens de Programação Computadores compreendem apenas código binário (0s e 1s), o que é inviável para humanos.

3

As linguagens de programação de alto nível (ex: Python, Java) permitem escrever instruções de forma mais intuitiva.

4

Essas instruções são então traduzidas para binário por interpretadores ou compiladores.

Primeiros Passos com Python e Fundamentos da Programação

Por que Aprender Python.

- O interpretador Python é o programa que lê e executa seu código.
- Um IDE (Ambiente de Desenvolvimento Integrado) ou Editor de Código oferece uma interface para escrever e gerenciar seus arquivos de código.
- Passos para Instalar Python Windows: - Baixe a versão estável mais recente (ex: 3.12.x) em [python.org/downloads/](https://www.python.org/downloads/).

Principais Aprendizados

1

Configurar um ambiente de desenvolvimento eficiente para programação em Python.

2

Criar e executar seu primeiro programa Python com sucesso.

3

A programação é a arte e a ciência de fornecer instruções a um computador para executar tarefas específicas.

4

Um programa de computador é uma sequência lógica de instruções que o computador pode entender.

MÓDULO 2

Estruturas de Dados Essenciais e Manipulação

O QUE VOCÊ VAI APRENDER

- Desenvolver a capacidade de manipular listas, tuplas, conjuntos e dicionários.
- Aplicar a estrutura de dados mais adequada para resolver problemas de armazenamento e organização de
- Fundamentos As estruturas de dados são formas organizadas de armazenar e gerenciar diferentes tip

Foto: Vitaly Gariev - Unsplash

Estruturas de Dados Essenciais e Manipulação

1. Desenvolver a capacidade de manipular listas, tuplas, conjuntos e dicionários.
2. Aplicar a estrutura de dados mais adequada para resolver problemas de armazenamento e organização de dados.
3. □ Fundamentos As estruturas de dados são formas organizadas de armazenar e gerenciar diferentes tipos de informações.
4. Em programação, assim como em um sistema de catalogação de livros, sua escolha é crucial para a eficiência e funcionalidade dos programas.

Estruturas de Dados Essenciais e Manipulação

1

Dominar estas estruturas permite ao programador resolver problemas complexos de forma elegante.

2

Permitem duplicatas e items de diferentes tipos (heterogêneas).

3

Criação e Acesso São definidas com colchetes [] e itens separados por vírgula.

4

Acessamos elementos usando indexação (lista[0]) ou fatiamento (lista[1:5]).

Estruturas de Dados Essenciais e Manipulação

Use .clear() para remover todos os elementos de uma vez, deixando a lista vazia.

- Modificar: Reatribua um valor a um índice específico: lista[indice] = novovalor (mylist[0] = 100).
- Iteração: Usamos laços for para percorrer os elementos, por exemplo, for elemento in minhalista: print(elemento).
- Também são ordenadas, permitem duplicatas e são heterogêneas.

Principais Aprendizados

1

Desenvolver a capacidade de manipular listas, tuplas, conjuntos e dicionários.

2

Aplicar a estrutura de dados mais adequada para resolver problemas de armazenamento e organização de dados.

3

 Fundamentos As estruturas de dados são formas organizadas de armazenar e gerenciar diferentes tipos de informações.

4

Em programação, assim como em um sistema de catalogação de livros, sua escolha é crucial para a eficiência e funcionalidade dos programas.

MÓDULO 3

Funções, Módulos e Organização de Código

O QUE VOCÊ VAI APRENDER

- Módulo 3: Funções, Módulos e Organização de Código Objetivo do Módulo Compreender a importância d
- Desenvolver a capacidade de criar e utilizar funções eficientes em Python.
- Aplicar princípios de modularização para construir programas mais organizados e reutilizáveis.

Foto: Fernando Garrido / Unsplash

Funções, Módulos e Organização de Código

1. **Módulo 3:** Funções, Módulos e Organização de Código □ Objetivo do Módulo Compreender a importância de funções, módulos e boa organização do código.

2. Desenvolver a capacidade de criar e utilizar funções eficientes em Python.

3. Aplicar princípios de modularização para construir programas mais organizados e reutilizáveis.

4. □ Fundamentos A programação eficaz se baseia na organização e reutilização do código.

Funções, Módulos e Organização de Código

1

Isso permite construir sistemas mais robustos e fáceis de manter.

2

Funções: Blocos de Código Reutilizáveis Uma função é um bloco de código que executa uma tarefa específica.

3

Ela pode ser chamada (executada) múltiplas vezes em diferentes partes do programa, promovendo a reutilização e modularidade.

4

Módulos: Agrupando Funcionalidades Um módulo é um arquivo Python que agrupa funções, variáveis e classes relacionadas.

Funções, Módulos e Organização de Código

> *Pense nele como uma "biblioteca" de funcionalidades que organiza o código em arquivos separados.*

- O corpo da função é indentado.
- nomedafuncao: Identificador único que deve descrever a ação da função (ex: calcularimpostopago), seguindo a convenção snakecase.
- parametro1, parametro2: Variáveis que recebem os valores passados quando a função é chamada (argumentos), cujo escopo é restrito ao corpo da função.

Principais Aprendizados

1

Módulo 3: Funções, Módulos e Organização de Código □ Objetivo do Módulo Compreender a importância de funções, módulos e boa organização do código.

2

Desenvolver a capacidade de criar e utilizar funções eficientes em Python.

3

Aplicar princípios de modularização para construir programas mais organizados e reutilizáveis.

4

□ Fundamentos A programação eficaz se baseia na organização e reutilização do código.

MÓDULO 4

Manipulação de Arquivos e Formatos de Dados (CSV, JSON)

O QUE VOCÊ VAI APRENDER

- Utilizar os módulos csv e json do Python para ler e gravar dados em arquivos, convertendo para e de
- Aplicar o conhecimento em cenários práticos que envolvem a troca de dados entre sistemas, configuraç
- É crucial para o desenvolvimento de software robusto.

Foto: Ana Tuha™ / Unsplash

Manipulação de Arquivos e Formatos de Dados (CSV, JSON)

1. Utilizar os módulos csv e json do Python para ler e gravar dados em arquivos, convertendo para e de estruturas de dados nativas.
2. Aplicar o conhecimento em cenários práticos que envolvem a troca de dados entre sistemas, configuração de aplicações e armazenamento de logs.
3. É crucial para o desenvolvimento de software robusto.
4. **Persistência de Dados:** Permite armazenar dados de forma permanente no disco rígido do sistema.

Manipulação de Arquivos e Formatos de Dados (CSV, JSON)

1

Diferente das variáveis em RAM que são temporárias e se perdem ao finalizar o programa, um arquivo garante a retenção da informação entre diferentes execuções.

2

Para interagir com um arquivo em Python, é preciso primeiro abrí-lo.

3

A função open() exige o nome do arquivo e o modo de abertura.

4

Escrita de conteúdo, começando do zero ou sobrescrevendo o existente.

Manipulação de Arquivos e Formatos de Dados (CSV, JSON)

> *Adicionar conteúdo ao final do arquivo existente.*

- Criação de um novo arquivo; falha se já existir.
- ⚠ Arquivo Existente Levanta FileNotFoundError se não existir.
- Truncado (conteúdo apagado) se existir.

Principais Aprendizados

1

Utilizar os módulos csv e json do Python para ler e gravar dados em arquivos, convertendo para e de estruturas de dados nativas.

2

Aplicar o conhecimento em cenários práticos que envolvem a troca de dados entre sistemas, configuração de aplicações e armazenamento de logs.

3

É crucial para o desenvolvimento de software robusto.

4

Persistência de Dados: Permite armazenar dados de forma permanente no disco rígido do sistema.

MÓDULO 5

Programação Orientada a Objetos (POO) com Python

O QUE VOCÊ VAI APRENDER

- Módulo 5: Programação Orientada a Objetos (POO) com Python Objetivo do Módulo Entender os conceit
- Aprender a criar classes e objetos em Python, definindo seus atributos e métodos.
- Aplicar os princípios da POO para desenvolver código mais organizado e reutilizável.

Foto: Samir Malek / Unsplash

Programação Orientada a Objetos (POO) com Python

1. **Módulo 5:** Programação Orientada a Objetos (POO) com Python □ Objetivo do Módulo Entender os conceitos fundamentais da Programação Orientada a Objetos (POO).

2. Aprender a criar classes e objetos em Python, definindo seus atributos e métodos.

3. Aplicar os princípios da POO para desenvolver código mais organizado e reutilizável.

4. □ Fundamentos A Programação Orientada a Objetos (POO) é um paradigma de programação que organiza o código em torno de objetos.

Programação Orientada a Objetos (POO) com Python

1

Essa abordagem permite modelar entidades do mundo real dentro do software.

2

Até então, programas foram desenvolvidos com lógica sequencial.

3

A POO facilita a gestão de programas com maior complexidade, ao segmentar o sistema em módulos independentes e interconectados.

4

Objetos combinam dados (atributos) e as funções (métodos) que operam sobre esses dados.

Programação Orientada a Objetos (POO) com Python

Classes Uma classe funciona como um molde ou projeto arquitetônico.

- Ela define as características (atributos) e os comportamentos (métodos) que os objetos daquele tipo terão.
- Em Python, a palavra-chave class é usada para defini-la.
- Exemplo: A classe Livro define que todo livro terá um título, autor e ISBN.

Principais Aprendizados

1

Módulo 5: Programação Orientada a Objetos (POO) com Python Objetivo do Módulo Entender os conceitos fundamentais da Programação Orientada a Objetos (POO).

2

Aprender a criar classes e objetos em Python, definindo seus atributos e métodos.

3

Aplicar os princípios da POO para desenvolver código mais organizado e reutilizável.

4

 Fundamentos A Programação Orientada a Objetos (POO) é um paradigma de programação que organiza o código em torno de objetos.

MÓDULO 6

Tratamento de Exceções e Geração de Logs

O QUE VOCÊ VAI APRENDER

- Módulo 6: Tratamento de Exceções e Geração de Logs Objetivo do Módulo Aplicar o tratamento de exc
- Utilizar a geração de logs para depurar, monitorar e manter aplicações Python.
- Desenvolver a capacidade de antecipar e gerenciar cenários de erro em sistemas computacionais.

Foto: Vitaly Gariev | Unsplash

Tratamento de Exceções e Geração de Logs

1. **Módulo 6:** Tratamento de Exceções e Geração de Logs □ Objetivo do Módulo Aplicar o tratamento de exceções para criar softwares robustos e tolerantes a falhas.

2. Utilizar a geração de logs para depurar, monitorar e manter aplicações Python.

3. Desenvolver a capacidade de antecipar e gerenciar cenários de erro em sistemas computacionais.

4. □ Fundamentos Em Python, uma exceção é um evento inesperado que interrompe o fluxo normal de execução de um programa.

Tratamento de Exceções e Geração de Logs

1

É um sinal de que algo deu errado durante a execução do código.

2

Se uma exceção não for devidamente tratada, o programa encerrará abruptamente, exibindo uma mensagem de erro ("traceback") no console.

3

O tratamento de exceções é um mecanismo para gerenciar esses erros de forma controlada.

4

Ele permite que o programa responda a condições excepcionais sem travar, oferecendo alternativas como exibir mensagens amigáveis ou registrar o problema.

Tratamento de Exceções e Geração de Logs

> *Diferente de erros de sintaxe, que são detectados antes da execução, as exceções ocorrem durante a execução do código.*

- A geração de logs é uma prática essencial para registrar informações sobre o comportamento do software.
- Isso inclui desde a depuração de problemas até o monitoramento em produção e a identificação de gargalos de desempenho.
- Juntos, o tratamento de exceções e a geração de logs são cruciais para a construção de softwares confiáveis e fáceis de manter.

Principais Aprendizados

1

Módulo 6: Tratamento de Exceções e Geração de Logs □ Objetivo do Módulo Aplicar o tratamento de exceções para criar softwares robustos e tolerantes a falhas.

2

Utilizar a geração de logs para depurar, monitorar e manter aplicações Python.

3

Desenvolver a capacidade de antecipar e gerenciar cenários de erro em sistemas computacionais.

4

□ Fundamentos Em Python, uma exceção é um evento inesperado que interrompe o fluxo normal de execução de um programa.

MÓDULO 7

Testes Automatizados com `unittest` e Boas Práticas

O QUE VOCÊ VAI APRENDER

- Fundamentos Testar softwares é crucial para garantir que funcionem como esperado.
- Os testes são verificações que confirmam o comportamento correto do código sob diversas condições.
- Os testes automatizados são executados repetidamente por um programa sem intervenção manual.

Foto: Patrick Amoy / Unsplash

Testes Automatizados com `unittest` e Boas Práticas

1. □ Fundamentos Testar softwares é crucial para garantir que funcionem como esperado.
2. Os testes são verificações que confirmam o comportamento correto do código sob diversas condições.
3. Os testes automatizados são executados repetidamente por um programa sem intervenção manual.
4. Eles contrastam com testes manuais, que são demorados e propensos a falhas humanas, exigindo até 40% mais tempo do QA para os mesmos resultados.

Testes Automatizados com `unittest` e Boas Práticas

1

Estima-se que o custo da correção de um bug em produção seja 100 vezes maior do que durante a fase de desenvolvimento inicial.

2

⚙ Como funciona O unittest é um módulo integrado do Python para escrever testes automatizados que aderem ao paradigma xUnit.

3

Inspirado no JUnit do Java, ele oferece uma abordagem orientada a objetos para criar e organizar suítes de teste.

4

Conceitos Essenciais do unittest Fixtures: Métodos que preparam o ambiente de teste e realizam a limpeza para garantir a independência.

Testes Automatizados com `unittest` e Boas Práticas

Casos de Teste (Test Cases): São classes que herdam de unittest.TestCase, agrupando testes relacionados a uma unidade específica de funcionalidade.

- Cada método que começa com test dentro dessas classes é um teste individual, isolando a verificação de um comportamento específico.
- Asserções (Assertions): Métodos que verificam condições específicas sobre o resultado de uma operação, determinando se um teste passa ou falha.

Testes Automatizados com `unittest` e Boas Práticas (Continuação)

> Se uma asserção falhar, o método de teste é imediatamente interrompido e marcado como falho, indicando um comportamento inesperado do código.

Principais Aprendizados

1

Fundamentos Testar softwares é crucial para garantir que funcionem como esperado.

2

Os testes são verificações que confirmam o comportamento correto do código sob diversas condições.

3

Os testes automatizados são executados repetidamente por um programa sem intervenção manual.

4

Eles contrastam com testes manuais, que são demorados e propensos a falhas humanas, exigindo até 40% mais tempo do QA para os mesmos resultados.

MÓDULO 8

Projeto Final: Construindo e Distribuindo um Utilitário Python

O QUE VOCÊ VAI APRENDER

- Módulo 8: Projeto Final: Construindo e Distribuindo um Utilitário Python Objetivo do Módulo Proje
- Empacotar e gerenciar dependências de projetos Python profissionalmente.
- Publicar e distribuir utilitários Python no PyPI para uso da comunidade.

Foto: Aalo Lens / Unsplash

Projeto Final: Construindo e Distribuindo um Utilitário Python

1

Módulo 8: Projeto Final: Construindo e Distribuindo um Utilitário Python □ Objetivo do Módulo Projetar e codificar um utilitário Python funcional do zero.

2

Empacotar e gerenciar dependências de projetos Python profissionalmente.

3

Publicar e distribuir utilitários Python no PyPI para uso da comunidade.

4

□ Fundamentos Um utilitário Python é um programa pequeno e focado que automatiza ou simplifica tarefas específicas.

Projeto Final: Construindo e Distribuindo um Utilitário Python

> *Ele pode ser um script de organização de arquivos, um conversor de unidades ou uma ferramenta de linha de comando.*

- A escolha de um utilitário para desenvolver neste projeto final permite a aplicação de conhecimentos em algo de interesse pessoal.
- O ciclo de desenvolvimento de um utilitário Python compreende desde a concepção da ideia até o empacotamento e distribuição do código.
- Distribuir um utilitário Pyhton significa torná-lo instalável e utilizável por outras pessoas com comandos simples.

Projeto Final: Construindo e Distribuindo um Utilitário Python

As ferramentas padrão da comunidade Python, como setuptools e PyPI, permitem essa distribuição.

⚙ Como funciona A construção de um utilitário Python segue um processo estruturado, desde a ideia inicial até a distribuição.

Cada etapa é fundamental para a criação de soluções eficientes e compartilháveis.

Definição e Planejamento do Utilitário Problema a ser resolvido: Determine a necessidade específica que o utilitário vai suprir de forma clara e mensurável.

Principais Aprendizados

1

Módulo 8: Projeto Final: Construindo e Distribuindo um Utilitário Python □ Objetivo do Módulo Projetar e codificar um utilitário Python funcional do zero.

2

Empacotar e gerenciar dependências de projetos Python profissionalmente.

3

Publicar e distribuir utilitários Python no PyPI para uso da comunidade.

4

□ Fundamentos Um utilitário Python é um programa pequeno e focado que automatiza ou simplifica tarefas específicas.

Obrigado!

Curso de introdução à Programação em Python

C O N C L

abril de 2

Foto: Alvaro Reyes / Unsplash